

Part V

Conclusion



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

You Mean There's More?

Congratulations! You've reached the end of your journey through the landscape of game engine architecture in one piece (and hopefully none the worse for wear). With any luck, you've learned a great deal about the major components that comprise a typical game engine. But of course, every journey's end is another's beginning. There's a great deal more to be learned about each and every topic covered within these pages. As technology and computing hardware continue to improve, more things will become possible in games—and more engine systems will be invented to support them. What's more, this book's focus was on the game engine itself. We haven't even begun to discuss the rich world of gameplay programming, a topic that could fill many more volumes.

In the following brief sections, I'll identify a few of the engine and gameplay systems we didn't have room to cover in any depth in this book, and I'll suggest some resources for those who wish to learn more about them.

17.1 Some Engine Systems We Didn't Cover

17.1.1 Movie Player

Most games include a movie player for displaying prerendered movies, also known as full-motion video (FMV). The basic components of the movie player

are an interface to the streaming file I/O system (see Section 7.1.3), a codec to decode the compressed video stream, and some form of synchronization with the audio playback system for the sound track.

A number of different video encoding standards and corresponding codecs are available, each one suited to a particular type of application. For example, video CDs (VCD) and DVDs use MPEG-1 and MPEG-2 (H.262) codecs, respectively. The H.261 and H.263 standards are designed primarily for online video conferencing applications. Games often use standards like MPEG-4 part 2 (e.g., DivX), MPEG-4 Part 10 / H.264, Windows Media Video (WMV) or Bink Video (a standard designed specifically for games by Rad Game Tools, Inc.). See http://en.wikipedia.org/wiki/Video_codec and <http://www.radgame.com/bnkmain.htm> for more information on video codecs.

17.1.2 Multiplayer Networking

Although the concepts of concurrent programming covered in Chapter 4 are relevant to multiplayer game architecture and distributed network programming, this book doesn't address either topic directly. For an in-depth treatment of multiplayer networking, see [4].

17.2 Gameplay Systems

A game is of course much more than just its engine. On top of the gameplay foundation layer (discussed in Chapter 16), you'll find a rich assortment of genre- and game-specific gameplay systems. These systems tie the myriad game engine technologies described in this book together into a cohesive whole, breathing life into the game.

17.2.1 Player Mechanics

Player mechanics are of course the most important gameplay system. Each genre is defined by a general style of player mechanics and gameplay, and of course every game within a genre has its own specific designs. As such, player mechanics is a huge topic. It involves the integration of human interface device systems, motion simulation, collision detection, animation and audio, not to mention integration with other gameplay systems like the game camera, weapons, cover, specialized traversal mechanics (ladders, swinging ropes, etc.), vehicle systems, puzzle mechanics and so on.

Clearly, player mechanics are as varied as the games themselves, so there's no one place you can go to learn all about them. It's best to tackle this topic by studying a single genre at a time. Play games and try to reverse-engineer

their player mechanics. Then try to implement them yourself! And as a very modest start on reading, you can check out [9, Section 4.11] for a discussion of Mario-style platformer player mechanics.

17.2.2 Cameras

A game's camera system is almost as important as the player mechanics. In fact, the camera can make or break the gameplay experience. Each genre tends to have its own camera control style, although of course every game within a particular genre does it a little bit differently (and some *very* differently). See [8, Section 4.3] for some basic game camera control techniques. In the following paragraphs, I'll briefly outline some of the most prevalent kinds of cameras in 3D games, but please note that this is far from a complete list.

- *Look-at cameras.* This type of camera rotates about a target point and can be moved in and out relative to this point.
- *Follow cameras.* This type of camera is prevalent in platformer, third-person shooter and vehicle-based games. It acts much like a look-at camera focused on the player character/avatar/vehicle, but its motion typically lags the player. A follow camera also includes advanced collision detection and avoidance logic and provides the human player with some degree of control over the camera's orientation relative to the player avatar.
- *First-person cameras.* As the player character moves about in the game world, a first-person camera remains affixed to the character's virtual eyes. The player typically has full control over the direction in which the camera should be pointed, either via mouse or joypad control. The look direction of the camera also translates directly into the aim direction of the player's weapon, which is typically indicated by a set of disembodied arms and a gun attached to the bottom of the screen, and a reticle at the center of the screen.
- *RTS cameras.* Real-time strategy and god games tend to employ a camera that floats above the terrain, looking down at an angle. The camera can be panned about over the terrain, but the pitch and yaw of the camera are usually not under direct player control.
- *Cinematic cameras.* Most three-dimensional games have at least some cinematic moments in which the camera flies about within the scene in a more filmic manner rather than being tethered to an object in the game. These camera movements are typically controlled by the animators.

17.2.3 Artificial Intelligence

Another major component of most character-based games is *artificial intelligence* (AI). At its lowest level, an AI system is usually founded in technologies like basic path finding (which commonly makes use of the well-known A* algorithm), perception systems (line of sight, vision cones, knowledge of the environment, etc.) and some form of memory or knowledge.

On top of these foundations, character control logic is implemented. A character control system determines how to make the character perform specific actions like locomoting, navigating unusual terrain features, using weapons, driving vehicles, taking cover and so on. It typically involves complex interfaces to the collision, physics and animation systems within the engine. Character control is discussed in detail in Section 12.10.

Above the character control layer, an AI system typically has goal setting and decision making logic, and possibly also emotional state modeling, group behaviors (coordination, flanking, crowd and flocking behaviors, etc.), and perhaps some advanced features like an ability to learn from past mistakes or adapt to a changing environment.

Of course, the term “artificial intelligence” is one of the biggest misnomers around in the game industry. Game AI is always more of a smoke and mirrors job than an attempt at truly mimicking human intelligence. Your AI characters might have all sorts of complex internal emotional states and finely tuned perception of the game world. But if the player cannot *perceive* the characters’ motivations, it’s all for naught.

AI programming is a rich topic, and we certainly have not done it justice in this book. For more information, see [18], [8, Section 3], [9, Section 3] and [47, Section 3]. Another good starting point is the GDC 2002 talk entitled, “The Illusion of Intelligence: The Integration of AI and Level Design in Halo,” by Chris Butcher and Jaime Griesemer of Bungie (<http://bit.ly/1g7FbhD>). And while you’re online, search for “game AI programming” too. You’ll find all sorts of links to talks, papers and books on game AI. The websites <http://aigamedev.com> and <http://www.gameai.com> are great resources as well.

17.2.4 Other Gameplay Systems

Clearly there’s a lot more to a game than just player mechanics, cameras and AI. Some games have drivable vehicles, implement specialized types of weaponry, allow the player to destroy the environment with the help of a dynamic physics simulation, let the player create his or her own characters, build custom levels, require the player to solve puzzles, Of course, the list of genre- and game-specific features, and all of the specialized software systems

that implement them, could go on forever. Gameplay systems are as rich and varied as games are. Perhaps this is where your next journey as a game programmer will begin!



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>